بسمه تعالی

تعریف: الگوریتم شایعه‌پراکنی یا Gossip Protocol روشی در سیستم‌های توزیع‌شده است که در آن گره‌ها (Node) اطلاعات را مانند پخش شدن شایعه بین خودشان منتشر می‌کنند. به این الگوریتم Epidemic Protocol (پروتکل همه‌گیر) هم گفته می‌شود، چون انتشار اطلاعات شبیه گسترش یک بیماری واگیردار است.

ایده‌ی اصلی: به جای اینکه یک گره مرکزی اطلاعات را به همه بدهد، هر گره در بازه‌های زمانی منظم (مثلاً هر یک ثانیه) چند گره‌ی تصادفی را انتخاب می‌کند و اطلاعات جدید را با آن‌ها مبادله می‌کند. گره‌های جدید هم همین کار را تکرار می‌کنند تا اطلاعات به کل شبکه برسد.

پارامترهای مهم:

۱.**Fanout**: تعداد گره‌هایی که هر گره در هر دور انتخاب می‌کند (مثلاً ۲ یا ۳).

۲.**بازه‌ی زمانی (Interval)**: فاصله‌ی بین دو دور ارسال.

۳.**تعداد دورها یا TTL**: مدت زمانی که یک خبر به انتشار ادامه می‌دهد تا شبکه با پیام تکراری پر نشود.

روش‌های تبادل اطلاعات:

روش	نحوه‌ی کار	ویژگی
Push	گرهی که خبر دارد، آن را برای گره‌های تصادفی می‌فرستد.	در ابتدای انتشار سریع است.
Pull	گره از گره‌های تصادفی می‌پرسد که خبر جدیدی دارند یا نه.	در مرحله‌ی پایانی که بیشتر گره‌ها خبر دارند، بهتر عمل می‌کند.
Push-Pull	هر دو گره اطلاعات خود را با هم مبادله می‌کنند.	سریع‌ترین روش است ولی پیام بیشتری مصرف می‌کند.

مراحل کار:

۱. گره‌ای اطلاعات جدیدی دریافت یا تولید می‌کند (مثلاً تغییر در تنظیمات).

۲. در دور بعدی، گره به تعداد fanout گره‌ی تصادفی پیام می‌فرستد.

۳. هر گره‌ی گیرنده، اگر خبر برایش تازه باشد، آن را ذخیره می‌کند و در دورهای بعد پخش می‌کند.

۴. اگر گره‌ای پیام تکراری دریافت کند، آن را نادیده می‌گیرد.

۵. این روند تا رسیدن خبر به تقریباً همه‌ی گره‌ها ادامه پیدا می‌کند.

مثال: فرض کنید ۸ سرور S۱ تا S۸ داریم و S۱ یک تنظیم جدید دریافت کرده است. fanout را ۲ در نظر بگیریم.

دور	اتفاق‌ها	گره‌های آگاه
۰	S۱ خبر را دارد.	S۱ (۱ سرور)
۱	S۱ برای S۳ و S۶ می‌فرستد.	S۱ و S۳ و S۶ (۳ سرور)
۲	S۱ برای S۲ و S۵، S۳ برای S۴ و S۸، S۶ برای S۷ و S۳ می‌فرستد (پیام S۳ تکراری است).	همه‌ی ۸ سرور

همان‌طور که دیده می‌شود، فقط با ۲ دور تمام سرورها مطلع شدند و هیچ گره‌ای مجبور نبوده به همه پیام بدهد. پیام تکراری هم طبیعی است و نادیده گرفته می‌شود.

سرعت انتشار:

در حالت ایده‌آل (بدون پیام تکراری) تعداد گره‌های آگاه در هر دور حدود (۱ + fanout) برابر می‌شود. برای مثال با fanout = ۲ تعداد گره‌ها به ترتیب ۱، ۳، ۹، ۲۷، ۸۱، ۲۴۳، ۷۲۹ می‌شود، یعنی برای حدود ۱۰۰۰ گره تنها حدود ۷ دور کافی است. به همین دلیل می‌گویند زمان انتشار تقریباً متناسب با لگاریتم تعداد گره‌ها ($O(\log N)$) است.

مزایا:

۱.مقیاس‌پذیری: با زیاد شدن گره‌ها، هر گره فقط با چند همسایه ارتباط دارد و بار روی یک گره زیاد نمی‌شود.

۲.تحمل خطا: خرابی چند گره یا گم شدن چند پیام باعث توقف انتشار نمی‌شود، چون مسیرهای زیادی وجود دارد.

۳.بدون نقطه‌ی شکست واحد: سرور مرکزی وجود ندارد که اگر از کار بیفتد، کل سیستم از کار بیفتد.

۴.سادگی: پیاده‌سازی آن ساده است و نیاز به هماهنگی پیچیده بین گره‌ها ندارد.

معایب:

۱.پیام تکراری: گره‌ها ممکن است یک خبر را چند بار دریافت کنند و پهنای باند هدر برود.

۲.احتمالاتی بودن: رسیدن خبر به همه‌ی گره‌ها تضمین قطعی ندارد و فقط با احتمال بسیار بالا اتفاق می‌افتد.

۳.سازگاری نهایی (Eventual Consistency): گره‌ها در یک لحظه‌ی مشخص ممکن است اطلاعات متفاوتی داشته باشند و فقط بعد از مدتی یکسان می‌شوند.

۴.تأخیر نامشخص: زمان دقیق رسیدن خبر به همه مشخص نیست.

مقایسه با روش‌های دیگر:

معیار	سرور مرکزی	پخش مستقیم (Broadcast)	شایعه‌پراکنی
بار روی یک گره	زیاد	زیاد	کم
نقطه‌ی شکست واحد	دارد	ندارد	ندارد
مقیاس‌پذیری	کم	کم	زیاد
تضمین رسیدن	قطعی‌تر	قطعی‌تر	احتمالاتی
مصرف پیام	کم	زیاد	متوسط

کاربردها:

۱.عضویت و کشف خرابی (Membership): هر گره از طریق شایعه‌پراکنی می‌فهمد کدام گره‌ها زنده یا خراب هستند. پروتکل SWIM که در ابزارهایی مثل Consul و Serf به کار رفته بر این پایه است.

۲.پایگاه‌های داده‌ی توزیع‌شده: Apache Cassandra برای انتشار اطلاعات وضعیت گره‌ها و Amazon Dynamo برای هماهنگی بین گره‌ها از این روش استفاده می‌کنند.

۳.انتشار تغییرات و پیکربندی: پخش تنظیمات جدید در یک خوشه‌ی بزرگ از سرورها.

۴.بلاکچین و شبکه‌های همتا به همتا: تراکنش‌ها و بلاک‌ها بین گره‌ها به شیوه‌ای مشابه شایعه‌پراکنی پخش می‌شوند.

۵.محاسبه‌ی مقادیر کلی (Aggregation): مثلاً میانگین بار سرورها بدون نیاز به سرور مرکزی.

نتیجه‌گیری: الگوریتم شایعه‌پراکنی با تکیه بر ارتباط تصادفی و تکراری بین گره‌ها، اطلاعات را در مدت کوتاهی در شبکه‌های بزرگ پخش می‌کند. این الگوریتم ساده، مقیاس‌پذیر و مقاوم در برابر خرابی است، ولی به قیمت پیام تکراری و سازگاری نهایی.